

Тема 2. Модули в Node.js



хекслет колледж



Цель занятия:

Познакомить студентов с системой модулей Node.js - научить разбивать программу на отдельные файлы, подключать встроенные модули и сторонние пакеты.

Учебные вопросы:

1. Что такое модуль и зачем он нужен
2. Подключение модулей - функция `require()`.
3. Встроенные модули Node.js
4. Сторонние модули

1. Что такое модуль и зачем он нужен

Проблема «одного большого файла»

Представим, что мы пишем программу и весь код держим в одном файле `index.js`.

Сначала это удобно - всё на виду. Но со временем файл разрастается: сотни, потом тысячи строк кода.

Найти нужное место становится сложно, а любое изменение может сломать что-то в другом месте.

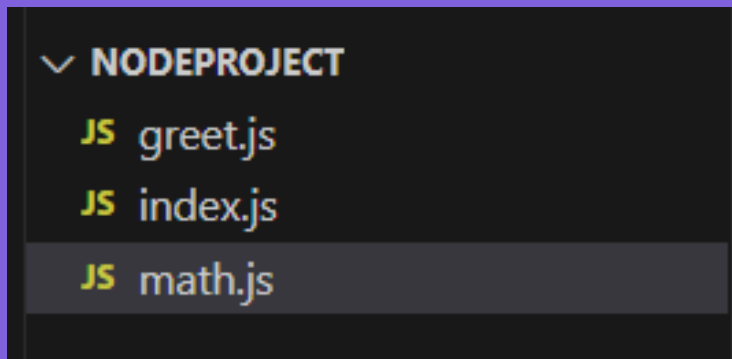
Пример:

```
1  // index.js – всё в одном месте
2  function greet(name) {
3    |   return `Привет, ${name}!`;
4  }
5
6  function add(a, b) {
7    |   return a + b;
8  }
9
10 function multiply(a, b) {
11   |   return a * b;
12 }
13
14 console.log(greet('Алексей'));
15 console.log(add(5, 3));
16 console.log(multiply(4, 3));
```

Пока функций три – всё нормально. Но когда их станет тридцать, такой файл превратится в хаос.

Что такое модуль?

Модуль - это отдельный файл с определённой задачей. Вместо одного большого файла программа разбивается на небольшие части, каждая из которых отвечает за что-то одно. Тот же пример, но разбитый на модули:



Каждый файл - отдельный модуль со своей задачей.

Преимущества модульного подхода:

- Читаемость - каждый файл небольшой и понятный
- Повторное использование - готовый модуль можно подключить в другом проекте
- Независимость - изменения в одном модуле не ломают остальные
- Командная работа - разные разработчики работают над разными модулями

Модули в Node.js

В Node.js каждый файл `.js` автоматически является модулем. По умолчанию всё что написано внутри файла - закрыто и недоступно снаружи.

Чтобы сделать что-то доступным - нужно явно это экспортировать. Чтобы использовать в другом файле - нужно импортировать.

Экспорт -> Импорт:

`math.js` => экспортирует функции => `index.js`
(использует функции)

Итог:

Модуль	Отдельный файл с определённой задачей
Зачем	Читаемость, повторное использование, независимость
В Node.js	Каждый .js файл — это модуль
По умолчанию	Всё внутри файла закрыто от других файлов

2. Подключение модулей - функция `require()`

Что такое `require()`

`require()` - это встроенная функция Node.js для подключения модулей.

Она принимает путь к модулю или его имя, загружает файл и возвращает то, что модуль экспортировал.

```
const math = require('./math');
```

Что можно подключить через require()

Тип модуля	Пример	Описание
Свой файл	<code>require('./math')</code>	Файл из проекта
Встроенный	<code>require('fs')</code>	Встроен в Node.js
Сторонний	<code>require('axios')</code>	Установлен через npm

Подключение своего файла

Создадим два файла:

math.js - модуль с функцией:

```
1  // math.js
2  function add(a, b) {
3    |   return a + b;
4  }
5
6  module.exports = add;
```

index.js - подключаем и используем:

```
1  const add = require('./math');  
2  
3  console.log(add(5, 3)); // 8
```

Путь начинается с ./ - это значит «в той же папке». Без ./ Node.js будет искать встроенный или сторонний модуль с таким именем.

`module.exports` - что это?

Когда Node.js загружает файл, каждый файл получает специальный объект `module`.

У этого объекта есть свойство `exports` - именно оно определяет, что будет доступно снаружи при подключении через `require()`.

По умолчанию `module.exports` - это пустой объект `{}`.

Пока мы ничего не присвоили - снаружи ничего не видно.

Чтобы функция стала доступна снаружи - присваиваем её в `module.exports`:

```
module.exports = add;
```

Если функций несколько - экспортируем их как объект:

```
// math.js
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

module.exports = { add, multiply };
...

```

index.js:

```
// index.js
const math = require('./math');

console.log(math.add(5, 3));      // 8
console.log(math.multiply(4, 3)); // 12
```

Или через деструктуризацию:

```
// index.js
const { add, multiply } = require('./math');

console.log(add(5, 3));      // 8
console.log(multiply(4, 3)); // 12
```

```
// index.js
const { add, multiply } = require('./math');

console.log(add(5, 3));      // 8
console.log(multiply(4, 3)); // 12
```

Результат тот же - это просто более короткая запись.

Сокращённая запись через exports
Вместо module.exports можно использовать
сокращение - просто exports:

```
// math.js
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

exports.add = add;
...
exports.multiply = multiply;
```

Простыми словами: `module.exports` - это «дверь» из модуля наружу.

Всё что в неё положили - будет доступно в другом файле через `require()`.

Всё остальное остаётся внутри и недоступно снаружи.

Расширение .js можно не писать

Node.js автоматически добавляет .js,
поэтому оба варианта равнозначны:

```
const add = require('./math.js'); // так  
const add = require('./math');    // и так — одно и то же
```

Порядок поиска модуля.

Когда вызывается `require('название')`, Node.js ищет модуль в следующем порядке:

- Встроенный модуль - например `fs`, `path`, `os`
- Файл в проекте - если путь начинается с `./` или `../`
- Папка `node_modules` - сторонние пакеты, установленные через `npm`

3. Встроенные модули Node.js

Встроенные модули - это модули, которые поставляются вместе с Node.js.

Их не нужно устанавливать через npm (Node Package Manager) - они уже есть. Достаточно подключить через require().

```
const fs = require('fs');  
const path = require('path');
```

Обратите внимание - путь не начинается с ./ . Это сигнал для Node.js что нужно искать встроенный или сторонний модуль, а не файл в проекте.

Основные встроенные модули

Модуль	Назначение
fs	Работа с файловой системой
path	Работа с путями к файлам и папкам
os	Информация об операционной системе
events	Работа с событиями
http	Создание веб-сервера

Пример:

```
const os = require("os")  
  
console.log(os.platform()); // win32 / linux / darwin  
console.log(os.arch());     // x64  
console.log(os.hostname()); // имя компьютера  
console.log(os.homedir());  // домашняя папка пользователя  
console.log(os.freemem());   // свободная память в байтах  
console.log(os.totalmem());  // всего памяти в байтах
```

Подробнее основные встроенные модули разберем в следующих лекциях.

4. Сторонние модули

Сторонние модули - это готовые пакеты, написанные другими разработчиками и опубликованные на npmjs.com.

Их не нужно писать самому - достаточно установить и использовать.

На сегодняшний день на npmjs.com опубликовано более 2 миллионов пакетов - на любой случай.

Установка стороннего модуля

Перед установкой любого пакета нужно инициализировать проект:

`npm init -y`

Эта команда создаст файл `package.json` - он хранит информацию о проекте и его зависимостях.

Затем установить нужный пакет:

`npm install название-пакета`

После установки появится папка `node_modules` - в ней хранятся все установленные пакеты.

```
▼ NODEPROJECT
  > node_modules
  JS index.js
  {} package.json
```

Пример - пакет chalk

chalk - популярный пакет для цветного вывода текста в терминале.

Установка:

```
PS C:\Users\user\Documents\nodeproject> npm init -y
```

```
PS C:\Users\user\Documents\nodeproject> npm install chalk@4
```

P.S: устанавливается chalk версии 4, т.к. начиная с версии 5 пакет перешёл на ES Modules и больше не поддерживает require().

Существует современный синтаксис import - познакомимся с ним позже.

Использование:

```
const chalk = require('chalk');  
  
console.log(chalk.green('Успех!'));  
console.log(chalk.red('Ошибка!'));  
console.log(chalk.yellow('Предупреждение!'));  
console.log(chalk.blue.bold('Информация'));
```

```
PS C:\Users\user\Documents\nodeproject> node index.js  
Успех!  
Ошибка!  
Предупреждение!  
Информация
```

Пример - пакет lodash

lodash - популярный пакет с набором вспомогательных функций для работы с массивами, объектами и строками.

Установка:

```
npm install lodash
```

Использование:

```
const _ = require('lodash');

const numbers = [1, 2, 3, 4, 5];

console.log(_.reverse([...numbers])); // [5, 4, 3, 2, 1]
console.log(_.sum(numbers));           // 15
console.log(_.max(numbers));           // 5
console.log(_.shuffle(numbers));       // [3, 1, 5, 2, 4] (случайный порядок)
```

Отличия встроенных и сторонних модулей

Модуль	Встроенные	Сторонние
Где находятся	Внутри Node.js	npmjs.com
Установка	Не нужна	<code>npm install</code>
Подключение	<code>require('fs')</code>	<code>require('chalk')</code>
Примеры	<code>fs</code> , <code>path</code> , <code>os</code>	<code>chalk</code> , <code>lodash</code>

Итог:

- Сторонние модули скачиваются с npmjs.com
- Перед установкой - инициализировать проект: `npm init -y`
- Установка пакета: `npm install название`
- После установки пакет появляется в папке `node_modules`
- Подключение - так же через `require()`, без `./`

Выводы:

- Модуль - это отдельный файл с определённой задачей
- Каждый .js файл в Node.js автоматически является модулем
- По умолчанию всё внутри файла закрыто - нужно явно экспортировать
- `module.exports` - определяет что будет доступно снаружи модуля
- `require()` - подключает модуль и возвращает то, что он экспортировал
- Путь к своему файлу начинается с `./`, встроенные и сторонние - без `./`
- Несколько значений экспортируются через объект: `module.exports = { add, multiply }`
- `exports.name = value` - сокращённая запись
- Встроенные модули уже есть в Node.js, устанавливать их не нужно
- Сторонние модули устанавливаются через `npm install`
- Перед установкой сторонних пакетов - инициализировать проект: `npm init -y`
- Все установленные пакеты хранятся в папке `node_modules`
- Некоторые пакеты (например `chalk 5+`) перешли на ES Modules и не работают с `require()` - в таком случае устанавливать более старую версию

Контрольные вопросы

1. Что такое модуль? Зачем разбивать программу на модули?
2. Как подключить свой файл через `require()`? Чем отличается от подключения встроенного модуля?
3. Что такое `module.exports`? Что будет, если ничего не экспортировать?
4. Как экспортировать из модуля несколько функций?
5. В чём разница между `module.exports` и `exports`? Почему нельзя писать `exports = значение`?
6. Сколько раз Node.js выполняет файл модуля, если он подключён в нескольких местах?
7. Назовите три встроенных модуля Node.js и их назначение.
8. Как установить сторонний пакет? Какую команду нужно выполнить перед установкой?
9. Где хранятся установленные пакеты?
10. Почему chalk версии 5 не работает с `require()`? Как решить эту проблему?